PRÁCTICA 4 + EXTRA 1 + EXTRA 2

January 1, 2024

Dado que el ejercicio principal es una continuación de la práctica 3 solo se han introducido pequeñas modificaciones en el código. Se presentará en este notebook el código completo para la presentación de los resultados y comentar escuetamente los principales añadidos.

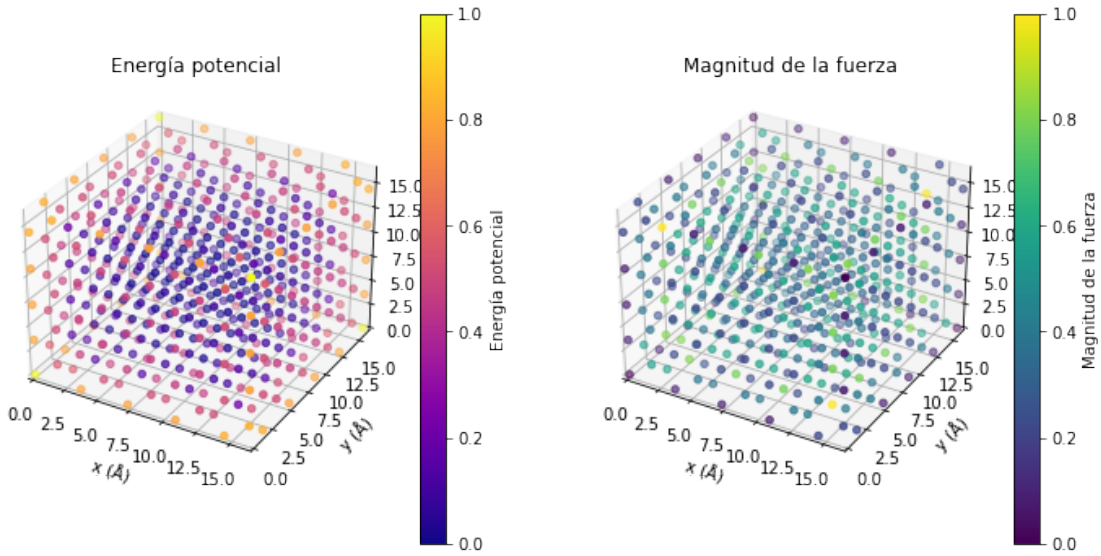
[1]:
```
import numpy as np # Para hacer cálculos numéricos
import matplotlib.pyplot as plt # Para hacer gráficos
from mpl_toolkits.mplot3d import Axes3D # Para hacer gráficos en 3D
import math # Para usar funciones matemáticas

# Definir las constantes y los parámetros del problema
sigma = 2.3151 # Parámetro de longitud del potencial de Lennard-Jones en Å
epsilon = 0.167 # Parámetro de energía del potencial de Lennard-Jones en eV
a = 3.603 # Parámetro de red de la red f.c.c. en Å
n = 12 # Exponente repulsivo del potencial
m = 6 # Exponente atractivo del potencial

# Definir una función que genere una red f.c.c. a partir del número de celdas␣
↪unidad en cada dirección y del parámetro de red
def generar_red_fcc(nx, ny, nz, a):
  # Crear una matriz vacía para almacenar las posiciones de los átomos
  N = 4 * nx * ny * nz # Número total de átomos
  pos = np.zeros((N, 3)) # Matriz de N filas y 3 columnas
  # Recorrer las celdas unidad en cada dirección
  i = 0 # Índice para contar los átomos
  for ix in range(nx):
    for iy in range(ny):
      for iz in range(nz):
        # Calcular las coordenadas de la esquina inferior izquierda de la celda␣
↪unidad
        x0 = ix * a
        y0 = iy * a
        z0 = iz * a
        # Añadir las posiciones de los cuatro átomos de la celda unidad
        pos[i] = [x0, y0, z0] # Átomo en la esquina
        i += 1
        pos[i] = [x0 + 0.5 * a, y0 + 0.5 * a, z0] # Átomo en el centro de la␣
↪cara xy
        i += 1
```

```
        pos[i] = [x0, y0 + 0.5 * a, z0 + 0.5 * a] # Átomo en el centro de la␣
↪cara yz
        i += 1
        pos[i] = [x0 + 0.5 * a, y0, z0 + 0.5 * a] # Átomo en el centro de la␣
↪cara xz
        i += 1
  # Devolver la matriz de posiciones
  return pos

# Definir una función que calcule la distancia entre dos átomos dados sus␣
↪índices en la red
def distancia(i, j, pos, nx, ny, nz, a, periodicas):
  # Extraer las posiciones de los átomos i y j
  xi, yi, zi = pos[i]
  xj, yj, zj = pos[j]
  # Calcular la distancia euclídea entre los átomos
  dx = xj - xi
  dy = yj - yi
  dz = zj - zi
  r = math.sqrt(dx**2 + dy**2 + dz**2)
  # Si se aplican condiciones periódicas, corregir la distancia según la imagen␣
↪más cercana
  if periodicas:
    Lx = nx * a # Longitud de la caja en la dirección x
    Ly = ny * a # Longitud de la caja en la dirección y
    Lz = nz * a # Longitud de la caja en la dirección z
    # Si la distancia es mayor que la mitad de la longitud de la caja, restar la␣
↪longitud de la caja
    if dx > 0.5 * Lx:
      dx -= Lx
    elif dx < -0.5 * Lx:
      dx += Lx
    if dy > 0.5 * Ly:
      dy -= Ly
    elif dy < -0.5 * Ly:
      dy += Ly
    if dz > 0.5 * Lz:
      dz -= Lz
    elif dz < -0.5 * Lz:
      dz += Lz
    # Recalcular la distancia corregida
    r = math.sqrt(dx**2 + dy**2 + dz**2)
  # Devolver la distancia
  return r
```

```python
# Definir una función que calcule la energía de interacción entre dos átomos␣
↪dados sus índices y sus posiciones en la red, usando el potencial de␣
↪Lennard-Jones y el radio de corte
def energia_interaccion(i, j, pos, nx, ny, nz, a, periodicas, sigma, epsilon, n,␣
↪m, rc):
  # Calcular la distancia entre los átomos i y j
  r = distancia(i, j, pos, nx, ny, nz, a, periodicas)
  # Si la distancia es menor que el radio de corte, calcular la energía de␣
↪interacción según el potencial de Lennard-Jones
  if r < rc:
    u = 4 * epsilon * ((sigma / r)**n - (sigma / r)**m) # Fórmula del potencial
    Fij = 24 * epsilon * ((2 * (sigma**12 / r**13)) - (sigma**6 / r**7))
  # Si la distancia es mayor o igual que el radio de corte, la energía de␣
↪interacción es cero
  else:
    u = 0
    Fij = 0
  # Devolver la energía de interacción
  return u, Fij

# Definir una función que calcule la energía total del sistema como la suma de␣
↪las energías de interacción de todos los pares de átomos, teniendo en cuenta␣
↪las condiciones periódicas o de superficie libre
def energia_total(pos, nx, ny, nz, a, periodicas, sigma, epsilon, n, m, rc):
  # Crear una variable para almacenar la energía total
  U = 0
  # Crear una lista vacía para almacenar la energía potencial de cada átomo
  U_lista = []
  F_lista = []
  # Recorrer todos los átomos
  N = len(pos) # Número total de átomos
  for i in range(N):
    # Crear una variable para almacenar la energía potencial del átomo i
    Ui = 0
    Fi = 0
    # Recorrer todos los átomos distintos de i
    for j in range(N):
      if i != j: # Evitar contar el mismo átomo
        # Calcular la energía de interacción entre los átomos i y j y sumarla a␣
↪la energía total y a la energía potencial del átomo i
        Uij, Fij = energia_interaccion(i, j, pos, nx, ny, nz, a, periodicas,␣
↪sigma, epsilon, n, m, rc)
        U += 0.5*Uij
        Ui += 0.5 * Uij # Se divide por dos para evitar contar dos veces la␣
↪misma interacción
        Fi += 0.5*Fij
```

```
    # Añadir la energía potencial del átomo i a la lista
    F_lista.append(Fi)
    U_lista.append(Ui)
  # Devolver la energía total y la lista de energías potenciales
  return U, U_lista, F_lista

# Definir una función que represente gráficamente la red en 3 dimensiones,␣
 ↪usando colores para indicar la energía potencial de cada átomo
def graficar_red(pos, U_lista, F_lista):
  # Crear una figura
  fig = plt.figure(figsize=(12, 6))

  # Crear el primer subplot para la energía potencial
  ax1 = fig.add_subplot(121, projection='3d')
  # Extraer las coordenadas x, y, z de la matriz de posiciones
  x = pos[:, 0]
  y = pos[:, 1]
  z = pos[:, 2]
  # Calcular el color de cada átomo según su energía potencial.
  c = (np.array(U_lista) - min(U_lista)) / (max(U_lista) - min(U_lista))
  # Representar los átomos como puntos de colores
  scatter1 = ax1.scatter(x, y, z, c=c, cmap='plasma')
  cbar1 = plt.colorbar(scatter1, ax=ax1)
  cbar1.set_label('Energía potencial')
  ax1.set_title('Energía potencial')
  # Crear el segundo subplot para la magnitud de la fuerza
  ax2 = fig.add_subplot(122, projection='3d')
  # Calcular el color de cada átomo según la magnitud de la fuerza.
  f = (np.array(F_lista) - min(F_lista)) / (max(F_lista) - min(F_lista))
  # Representar los átomos como puntos de colores
  scatter2 = ax2.scatter(x, y, z, c=f, cmap='viridis')
  cbar2 = plt.colorbar(scatter2, ax=ax2)
  cbar2.set_label('Magnitud de la fuerza')
  ax2.set_title('Magnitud de la fuerza')
  # Ajustar los límites de los ejes y etiquetarlos para ambos subplots
  for ax in [ax1, ax2]:
    ax.set_xlim(min(x), max(x))
    ax.set_ylim(min(y), max(y))
    ax.set_zlim(min(z), max(z))
    ax.set_xlabel('x (Å)')
    ax.set_ylabel('y (Å)')
    ax.set_zlabel('z (Å)')

  # Mostrar la figura
  plt.show()
# Pedir al usuario que introduzca los datos de entrada
nx = int(input("Introduzca el número de celdas unidad en la dirección x: "))
```

```python
ny = int(input("Introduzca el número de celdas unidad en la dirección y: "))
nz = int(input("Introduzca el número de celdas unidad en la dirección z: "))
condiciones = input("Introduzca el tipo de condiciones de contorno (periódicas o␣
↪superficies libres): ")
rc = float(input("Introduzca el valor del radio de corte del potencial en␣
↪unidades de sigma: ")) * sigma
# Convertir el tipo de condiciones a un valor booleano
periodicas = condiciones.lower() == "periodicas" or condiciones.lower() ==␣
↪"periódicas"
# Verificar si el usuario ha introducido periódicas o superficies libres
if not periodicas:
  if condiciones.lower() != "superficies libres":
    print("Por favor, introduzca 'periódicas' o 'superficies libres' como tipo␣
↪de condiciones de contorno.")
    print('En caso de que no se introduzca un tipo de condiciones de contorno␣
↪válido, se asumirán condiciones de superficies libres.')
    condiciones = input("Introduzca el tipo de condiciones de contorno␣
↪(periódicas o superficies libres): ")
    periodicas = condiciones.lower() == "periódicas" or condiciones.lower() ==␣
↪"periodicas"
# Aplicar las funciones definidas anteriormente para generar la red, calcular la␣
↪energía total y mostrar el gráfico con colores
pos = generar_red_fcc(nx, ny, nz, a)
U, U_lista, F_lista = energia_total(pos, nx, ny, nz, a, periodicas, sigma,␣
↪epsilon, n, m, rc)
graficar_red(pos, U_lista, F_lista)
# Mostrar la energía total por átomo
print(f"La energía total del sistema por átomo es {U / len(pos)} eV.")
if not periodicas:
  condiciones = "superficies libres"
print('Se eligió como condiciones de contorno:', condiciones)
```




```
La energía total del sistema por átomo es -1.0714817373844845 eV.
Se eligió como condiciones de contorno: superficies libres
```

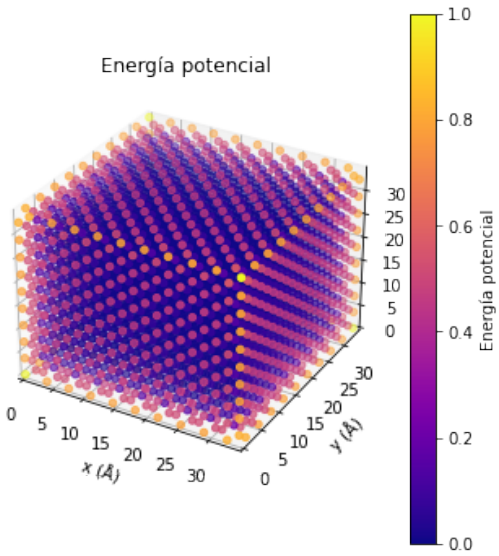
Podemos observar los resultados para el caso de 5 celdas en cada unidad y un radio de corte de 3 veces sigma. Se ha añadido otra representación tridimensional para el módulo de la fuerza de cada átomo, ahora en otra gama de colores y siguiendo la misma normalización para la escala.

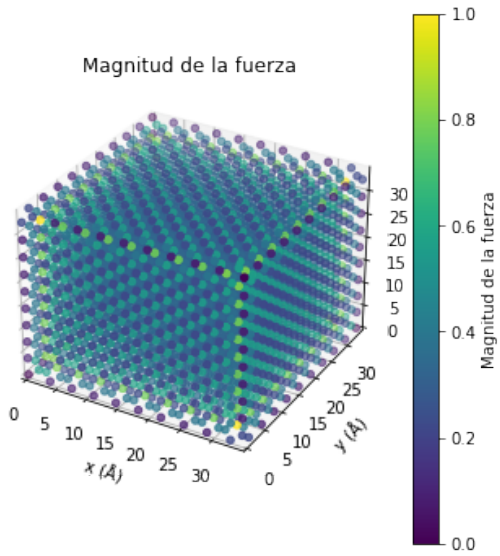
Observaremos resultados parecidos al los obtenidos en el ejercicio extra de la práctica anterior. Esto tiene fundamento pues la fuerza es proporcional al gradiente del potencial.

En cuanto a las modificaciones se refiere, se ha añadido un cálculo idéntico, al de la energía, en el interior de la función `energia_total` y `energia_interaccion`. Para ello, se añade de forma explicita la derivada del potencial y los valores de las constantes de las que depende.

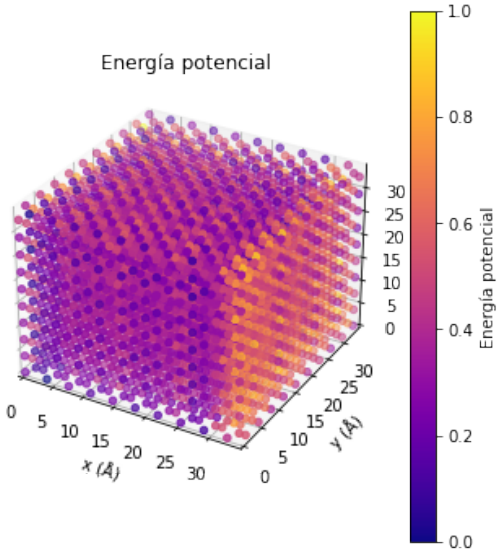
Por último, al final del código, en la función para graficar la red, se ha modificado para que nos muestre en una misma figura las dos representaciones tridimensionales. Una para los colores asociados a la energía y otro para los colores asociados a las fuerzas.

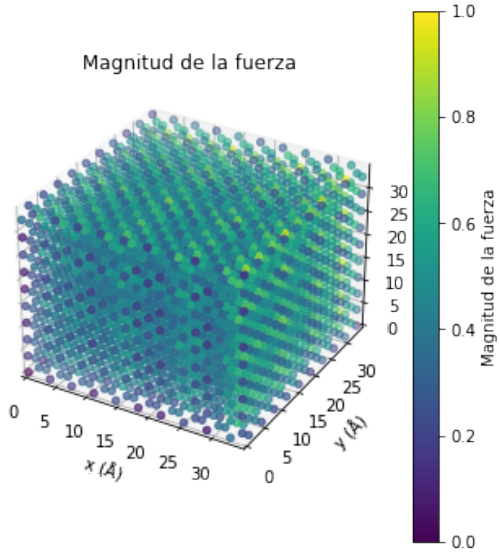
RED DE 10 CELDAS POR DIRECCIÓN. RADIO DE CORTE 3 SIGMA. SUPERFICIES LIBRES.





RED DE 10 CELDAS POR DIRECCIÓN. RADIO DE CORTE 3 SIGMA. PERIÓDICAS.





EXTRA 1

Para el desarrollo de los dos apartados extra de la práctica 4 nos ceñiremos, salvo excepciones, a seguir los puntos dados en cada uno de los apartados.

Definimos primeramente numoy como np y algunas constante a utilizar más adelante. Además, en cuanto al primer punto, pedimos al usuario que introduzca un valor de temperatura en kelvin.

[1]:
```python
# Importamos numpy para generar números aleatorios y hacer cálculos
import numpy as np

# Definimos la constante de Boltzmann en eV/K
KB = 8.6181024e-5

# Definimos la masa del cobre en uma
M = 63.55

# Pedimos al usuario que introduzca la temperatura deseada en K
Temp = float(input("Introduce la temperatura deseada en K: "))
```

Se define el número de particulas en 10000, se puede cambiar en el script si se quisiera. Además, se define una matriz con tres columnas, una para cada dirección, y N filas, tantas como partículas, donde se asignarán valores entre -0.5 y 0.5, de velocidad, para cada dirección de cada una de las partículas.

Una vez generada la matriz, se muestra por la terminal. Debido al gran número de filas, en este notebook eliminaremos dicha salida en el archivo .tex. Sin embargo, si se inicia el código le aparecerá dicha salida por terminal.

Se define la energía cinética y se tiene en cuenta las unidades. Pasamos la masa del cobre a kilogramos. Con ello, se nos queda la expresión Ec. En ella, se calcula la energía cinética del sistema (de cada una de las partículas), en eV. El argumento `axis = 1` indica que se suma a lo largo de las columnas.

[2]:
```python
N = 10000

# Generamos una matriz de N x 3 con valores aleatorios entre -0.5 y 0.5
# Cada fila representa una partícula y cada columna una componente de su␣
↪velocidad
V = np.random.uniform(-0.5, 0.5, (N, 3))
print(f'Las velocidades serán: {V}')
# Calculamos la energía cinética del sistema en eV
# Usamos la fórmula Ec = 1/2 * M * v**2, donde v es la magnitud de la velocidad
# Convertimos la masa de uma a kg multiplicando por 1.66054e-27
Ec = 0.5 * M * 1.66054e-27 * np.sum(V**2, axis=1)
```

```
Las velocidades serán: [[-0.03818007 -0.33868113  0.4399268 ]
 [-0.48213493  0.07196305  0.47982126]
 [-0.16019764 -0.40884872  0.19177842]
 ...
 [ 0.45835968 -0.34230299  0.20541696]
 [ 0.20566704 -0.47661882  0.34256235]
 [-0.01625969  0.40735391 -0.23570465]]
```

Se calcula Trandom, obtenida a partir de la distribución aleatoria de velocidades. Para observar cada uno de los pasos, siempre imprimimos por terminal cada una de las variables calculadas.

Se calcula, con esta Trandom, un factor de escala `sc` con la temperatura introducida por el usuario

al inicio.

Por último se escala la velocidad multiplicando por el factor `sc`.

[3]:
```
Trandom = 2/3 * np.sum(Ec) / (KB * N)
print(f'La temperatura random es: {Trandom}')
# Calculamos el factor de escala para ajustar la temperatura
# Usamos la fórmula sc = sqrt(Temp / Trandom)
sc = np.sqrt(Temp / Trandom)
print(f'El factor de escala: {sc}')
# Escalamos las velocidades multiplicando por el factor de escala
V = V * sc
```

```
La temperatura random es: 1.0233573893427401e-22
El factor de escala: 1712170312301.6099
```

Con las nuevas velocidades, modificadas con el factor de escala, se recalculan la energía cinética y la temperatura. Se imprime por terminal el recalculado valor de T para comprobar que obtenemos de nuevo la misma temperatura introducida. Así, nos aseguramos que lo realizado está bien.

[4]:
```
Ec = 0.5 * M * 1.66054e-27 * np.sum(V**2, axis=1)
T = 2/3 * np.sum(Ec) / (KB * N)

# Imprimimos el valor de la temperatura final
print(f"La temperatura querida para el sistema es {T:.2f} K")
```

```
La temperatura querida para el sistema es 300.00 K
```

En cuanto al reescalado de las velocidades en una simulación de dinámica molecular, es útil para controlar la temperatura del sistema. En una simulación de dinámica molecular, la temperatura es proporcional a la energía cinética media de las partículas, que a su vez es proporcional al cuadrado de la magnitud de la velocidad. Por lo tanto, si quieres aumentar o disminuir la temperatura del sistema, puedes hacerlo aumentando o disminuyendo las velocidades de las partículas.

El reescalado de las velocidades se hace a menudo en el proceso de equilibrado de la simulación. Durante el equilibrado, se realiza una serie de pasos de simulación en los que se reescalan las velocidades para llevar el sistema a la temperatura deseada. Esto ayuda a asegurar que el sistema está en el estado correcto antes de comenzar la parte principal de la simulación.

En este código, el reescalado de las velocidades se hace calculando la energía cinética actual del sistema, comparándola con la energía cinética que correspondería a la temperatura deseada, y luego multiplicando las velocidades por la raíz cuadrada de la relación de estas dos energías. Esto asegura que después del reescalado, la energía cinética media del sistema corresponderá a la temperatura deseada.

EXTRA 2

Para este último extra, se ha reciclado el extra anterior. Dado que en este caso, tendremos una mayor carga computacional y los tiempos pueden aumentar dado un número considerado de pasos Monte Carlo, se han introducido pequeñas modificaciones a la hora de realizar pequeños cálculos. Por ejemplo, a la hora de calcular la energía cinética, se ha reemplazado $np.sum(v^2)$

por $np.linalg.norm(v)^2$. También, para el calculo de valores medios, directamente se calculo con $np.mean()$.

Para este último extra tenemos que notar algo. La energía potencial que se ha considerado es la asociada a un potencial armónico. He implementado este potencial ya que para el de Lennard-Jones he tenido problemas a la hora de obtener los resultados.

No obstante cabe mencionar, que para el caso de una red cristalina, el considerar una interacción armónica entre los átomos vecinos no es nada descabellado. No obstante, puede que sea más apropiado el uso del potencial de la práctica 3, de Lennard-Jones, ya que este modela las interacciones de van der Waals y la repulsión de corto alcance entre átomos, ya que en este caso, al seleccionarse las velocidades y las posiciones de manera aleatoria estamos más cerca del modelado de un sistema de átomos o moléculas que no están unidos por enlaces químicos, como puede ser el caso de un gas o un líquido.

Puede parecer que los resultados no tienen que ser muy diferentes tomado un potencial u otro pero la verdad es que la elección del potencial afectará a las trayectorias de las partículas, y por tanto, a todas las propiedades del sistema que dependa de las trayectorias. Aquí podemos incluir la energía total del sistema, la temperatura, la presión, la difusión, la estructura, etc.

Por tanto, considerando lo comentado decir que el potencial que se va a implementar no es el más adecuado pero es con el que he conseguido obtener resultados. Al implementar el potencial de Lennard-Jones los tiempo de ejecución para obtener resultados para un número de partículas y de pasos significativos eran considerablemente elevados con lo que al final la elección tomada ha sido presentar este script mencionando los incovenientes. A pesar de esto, la ventaja de estar utilizando un potencial tan simple es que su implementación es muy sencilla y no nos supone mucha carga computacional.

[2]:
```
# Importamos numpy para generar números aleatorios y hacer cálculos
import numpy as np

# Importamos matplotlib para hacer gráficos
import matplotlib.pyplot as plt

# Definimos la constante de Boltzmann en eV/K
KB = 8.6181024e-5

# Definimos la masa del cobre en uma
M = 63.55

# Pedimos al usuario que introduzca la temperatura deseada en K
Temp = float(input("Introduce la temperatura deseada en K: "))

# Definimos el número de partículas
N = 10000

# Generamos una matriz de N x 3 con valores aleatorios entre -0.5 y 0.5
# Cada fila representa una partícula y cada columna una componente de su␣
↪velocidad
```

```python
V = np.random.uniform(-0.5, 0.5, (N, 3))
print(f'Las velocidades serán: {V}')
# Calculamos la energía cinética del sistema en eV
# Usamos la fórmula Ec = 1/2 * M * v**2, donde v es la magnitud de la velocidad
# Convertimos la masa de uma a kg multiplicando por 1.66054e-27
# Usamos la función np.linalg.norm para calcular la magnitud de la velocidad
Ec = 0.5 * M * 1.66054e-27 * np.linalg.norm(V, axis=1)**2

# Calculamos la temperatura del sistema en K
# Usamos la fórmula Trandom = 2/3 * Ec / (KB * N)
# Usamos la función np.mean para calcular la media de la energía cinética
Trandom = 2/3 * np.mean(Ec) / KB
print(f'La temperatura random es: {Trandom}')
# Calculamos el factor de escala para ajustar la temperatura
# Usamos la fórmula sc = sqrt(Temp / Trandom)
# Usamos la función np.sqrt para calcular la raíz cuadrada
sc = np.sqrt(Temp / Trandom)
print(f'El factor de escala: {sc}')
# Escalamos las velocidades multiplicando por el factor de escala
V = V * sc

# Volvemos a calcular la energía cinética y la temperatura del sistema
Ec = 0.5 * M * 1.66054e-27 * np.linalg.norm(V, axis=1)**2
T = 2/3 * np.mean(Ec) / KB

# Imprimimos el valor de la temperatura final
print(f"La temperatura querida para el sistema es {T:.2f} K")
```

```
Las velocidades serán: [[ 0.3309593   0.33423045  0.18270498]
 [ 0.22502397 -0.12941056 -0.00996651]
 [-0.12389767 -0.28821311 -0.466608  ]
 ...
 [ 0.39816473 -0.47147444  0.48306456]
 [ 0.24883719 -0.41827869 -0.42756387]
 [-0.12141697  0.33813909  0.46102907]]
La temperatura random es: 1.0222396429600834e-22
El factor de escala: 1713106124822.9697
La temperatura querida para el sistema es 300.00 K
```

Hasta esta parte, el código es idéntico al anterior. Se realizan el reescalado de velocidades a partir de las temperaturas calculadas e introducidas.

A continuación, se le pide al usuario que introduzca el número de iteraciones que se requiere para la simulación.

Se define el paso temporal pedido por el enunciado y se genera de forma aleatoria las posiciones de las partículas entre -5 y 5 Amstroms. De nuevo, se muestra una matriz con todas las posiciones, que en el documento .tex se eliminarán, al igual que hicimos con las velocidades en el anterior extra.

Ya por último, se define el potencial armónico y una serie de listas, las cuales iremos rellenando en cada iteración para la representación posterior.

[3]:
```
Nsteps = int(input("Introduce el número total de pasos de simulación: "))
print(f'El número total de pasos de simulación es: {Nsteps}')
# Definimos el paso de tiempo en s
h = 1e-15

# Generamos una matriz de N x 3 con valores aleatorios entre -5 y 5
# Cada fila representa una partícula y cada columna una componente de su posición
# Convertimos las posiciones de A a m multiplicando por 1e-10
X = np.random.uniform(-5, 5, (N, 3)) * 1e-10
print(f'Las posiciones serán: {X}')

# Calculamos la energía potencial del sistema en eV
# Usamos la fórmula Ep = 1/2 * k * x**2, donde x es la magnitud de la posición
# y k es la constante de fuerza en eV/m**2
# Asumimos que k = 1 eV/m**2
k = 1
Ep = 0.5 * k * np.linalg.norm(X, axis=1)**2

# Calculamos la energía potencial inicial del sistema
Epi = np.sum(Ep)

# Creamos listas vacías para almacenar la temperatura, la energía cinética, la␣
↪energía potencial y la energía total en cada paso
T_list = []
Ec_list = []
Ep_list = []
E_list = []

# Añadimos los valores iniciales a las listas
T_list.append(T)
Ec_list.append(np.sum(Ec))
Ep_list.append(np.sum(Ep) - Epi)
E_list.append(np.sum(Ec) + np.sum(Ep))
```

```
El número total de pasos de simulación es: 1000
Las posiciones serán: [[-2.67681424e-10  1.75395315e-10 -3.49623711e-10]
 [-4.58582031e-12  2.26859639e-10 -1.03085986e-10]
 [ 4.08790121e-10  1.85233808e-10  2.83961595e-10]
 ...
 [ 2.50483331e-10 -3.19232983e-10  4.69832153e-10]
 [-1.97472062e-10 -2.98877874e-10 -4.29077906e-11]
 [-3.34896391e-10  3.94416629e-10 -1.66041914e-10]]
```

Finalmente, aplicamos Verlet y realizamos los calculos de forma matricial, en cada paso actualizamos tanto la matriz de posiciones como la de velocidades. Se calcula, de igual manera, cada

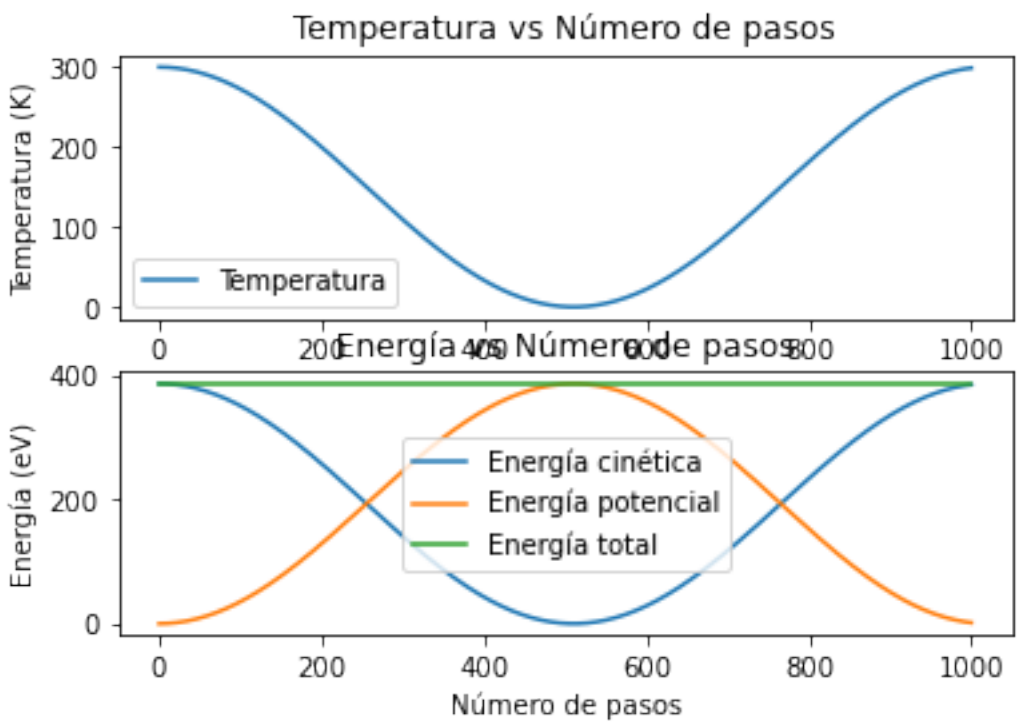
una de las energías, cinética y potencial, en cada paso Monte Carlo y se añaden a sus respectivas listas. Además, se calcula también la temperatura del paso.

[4]:
```
for i in range(Nsteps):
  # Calculamos las nuevas posiciones usando la fórmula X(t + h) = X(t) + V(t) *␣
 ↪h + 1/2 * a(t) * h**2
  # Asumimos que a(t) = -k * X(t) / M
  # Convertimos la masa de uma a kg multiplicando por 1.66054e-27
  X_new = X + V * h - 0.5 * k * X / (M * 1.66054e-27) * h**2

  # Calculamos las nuevas velocidades usando la fórmula V(t + h) = V(t) + 1/2 *␣
 ↪(a(t) + a(t + h)) * h, derivada de la expresión anterior
  # Asumimos que a(t + h) = -k * X(t + h) / M
  V_new = V - 0.5 * k * (X + X_new) / (M * 1.66054e-27) * h

  # Actualizamos las posiciones y las velocidades
  X = X_new
  V = V_new

  # Calculamos la nueva energía cinética
  Ec = 0.5 * M * 1.66054e-27 * np.linalg.norm(V, axis=1)**2

  # Calculamos la nueva energía potencial
  Ep = 0.5 * k * np.linalg.norm(X, axis=1)**2

  # Calculamos la nueva temperatura
  T = 2/3 * np.mean(Ec) / KB

  # Añadimos los nuevos valores a las listas
  T_list.append(T)
  Ec_list.append(np.sum(Ec))
  Ep_list.append(np.sum(Ep) - Epi)
  E_list.append(np.sum(Ec) + np.sum(Ep))
```

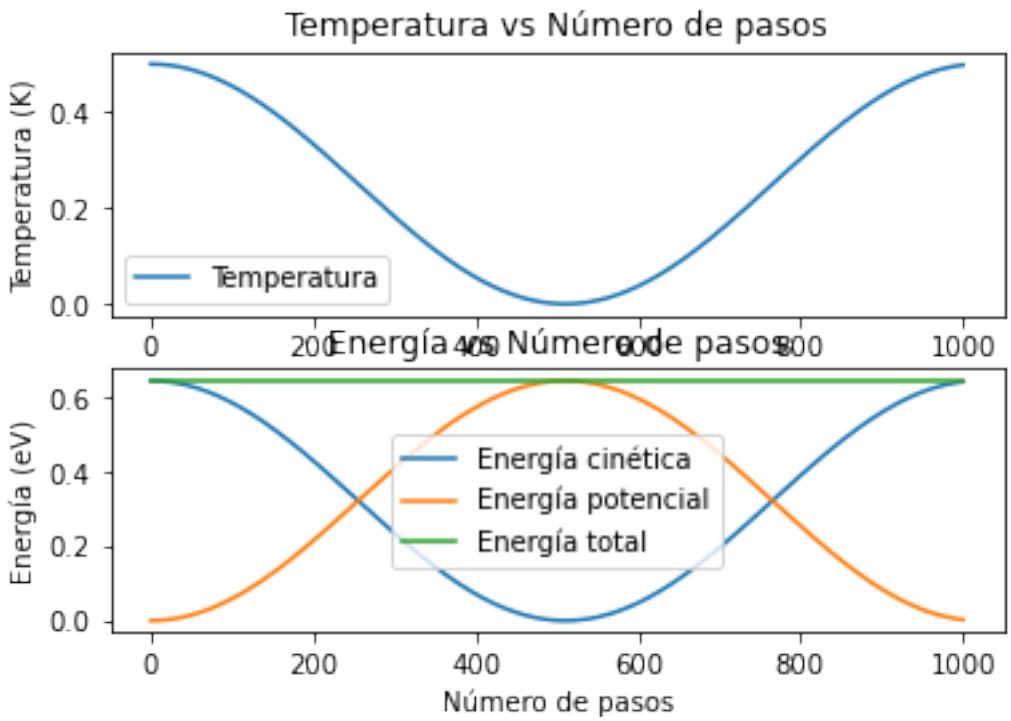
Por último, nos queda representar los resultados en función de los pasos Monte Carlo. Veamos diferentes casos.

[5]:
```
plt.figure()

# Representamos la temperatura en función del número de pasos de simulación en␣
 ↪el primer subplot
plt.subplot(2, 1, 1)
plt.plot(range(Nsteps + 1), T_list, label="Temperatura")
plt.xlabel("Número de pasos")
plt.ylabel("Temperatura (K)")
plt.title("Temperatura vs Número de pasos")
plt.legend()
```

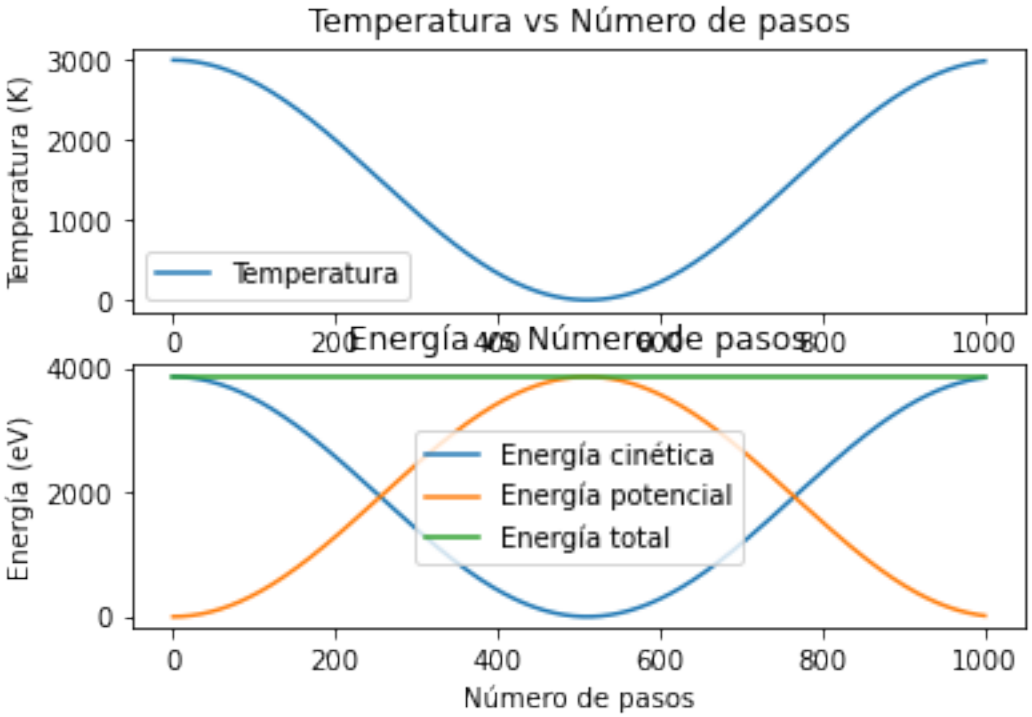
```
# Representamos la energía cinética, la energía potencial y la energía total en␣
↪función del número de pasos de simulación en el segundo subplot
plt.subplot(2, 1, 2)
plt.plot(range(Nsteps + 1), Ec_list, label="Energía cinética")
plt.plot(range(Nsteps + 1), Ep_list, label="Energía potencial")
plt.plot(range(Nsteps + 1), E_list, label="Energía total")
plt.xlabel("Número de pasos")
plt.ylabel("Energía (eV)")
plt.title("Energía vs Número de pasos")
plt.legend()

# Mostramos la figura
plt.show()
```



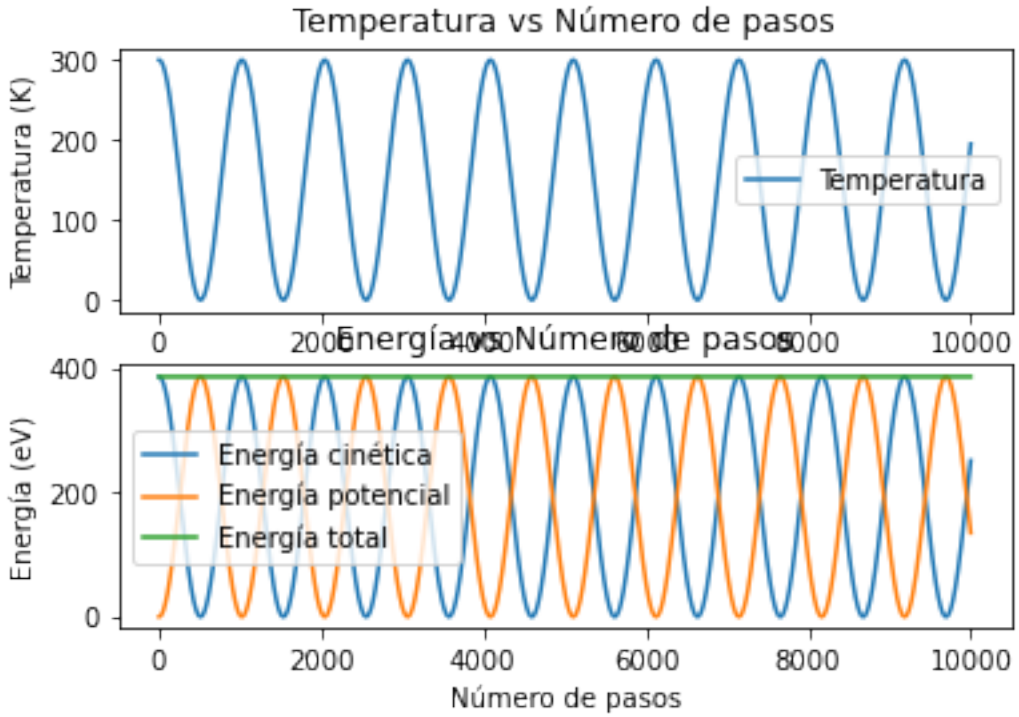
T = 0.5 K y número de pasos MC 1000



T = 3000 K y número de pasos MC 1000



T = 300 K y número de pasos MC 10000



En los resultados podemos observar como cambian en función del cambio de temperatura. Si introducimos temperaturas menores o mayores, dejando fijo en número de pasos, lo que le ocurre al sistema es que este oscila de la misma forma. El cambio se produce en el máximo de temperatura y energías que se alcanzan. Estas son proporcionales, con lo que tiene sentido. A mayor temperatura mayor altura alcanzan las curvas de energía.

IMPLEMENTACIÓN DEL POTENCIAL LENNARD-JONES

Para la implementación de dicho potencial utilicé una estructura parecida a la que muestro a continuación.

[1]:
```
# Calcular la energía potencial del sistema
Ep = 0
for i in range(N):
    for j in range(i+1, N):
        # Calcular la distancia entre los átomos i y j
        r = np.linalg.norm(X[i] - X[j])
        # Calcular el potencial de Lennard-Jones entre los átomos i y j
        V = 4 * epsilon * ((sigma / r)**12 - (sigma / r)**6)
        # Sumar el potencial de Lennard-Jones a la energía potencial total
        Ep += V
```